

Library Management System

Project Report

VITyarthi — Build Your Own Project

Course Submission (Flipped Course Evaluation)

Submitted by: [Your Name]

Registration Number: [Your Registration Number]

Course / Subject: [Course Name]

Date: September 26, 2026

Technology Stack: Python, Flask, SQLite

2. Introduction

Libraries and small reading rooms frequently rely on paper registers or spreadsheets to track their book inventory and lending activity. This project implements a lightweight, web-based Library Management System using Python's Flask framework and a SQLite database. It allows a librarian to manage a book catalog, issue and return books, and view a dashboard of catalog and lending statistics, all through a browser.

The project was built to satisfy the VITyarthi "Build Your Own Project" course requirements: it applies core subject concepts (web request/response handling, relational data modeling, CRUD operations, authentication, and automated testing) to a real-world problem, with full supporting design documentation.

3. Problem Statement

Without a dedicated system, library staff cannot quickly answer basic operational questions: which copies of a title are currently available, which loans are overdue, or who currently holds a given book. Manual records are slow to update, easy to lose, and provide no validation (e.g. nothing stops a copy count from going negative, or a book from being "issued" when no copies remain). A small, purpose-built application removes this friction for librarians managing a modest catalog.

4. Functional Requirements

The system implements the following functional modules (exceeding the minimum of three required):

- **User management** — librarian registration and login, with hashed passwords and session-based access control.
- **Book catalog (CRUD)** — add, edit, remove, and search books by title, author, or ISBN.
- **Lending workflow** — issue a book to a borrower with an automatic due date, and process returns.
- **Reporting / dashboard** — catalog totals, active loan counts, and an overdue-loans report.

Each module has a clear input/output structure: HTML forms collect input, the Flask route validates and processes it via the data-access layer, and the result (success, or a specific validation error) is displayed back to the user via flash messages and updated page state.

5. Non-functional Requirements

Requirement	How it's addressed
Security	Passwords hashed with PBKDF2 (werkzeug.security); every management route requires an active login
Reliability	All data persists in SQLite (not in-memory), with foreign keys and CHECK constraints preventing invalid
Usability	Consistent navigation and immediate flash-message feedback on every action.
Maintainability	Code split into single-responsibility modules (auth, books, loans, reports, data access, validation) with d

Error handling	Centralized ValidationError type, try/except around form processing, and custom 404/500 handlers.
Performance	Indexes on loans.book_id, loans.status, and books.title keep lookups fast as the catalog grows.

6. System Architecture

The app follows a layered architecture. Jinja2 templates render the UI in the browser; Flask blueprints handle routing per feature area (auth, books, loans, reports); a shared validation/access-control module (utils.py) is used by every write-oriented route; and a single data-access module (models.py) is the only place that talks to SQLite.

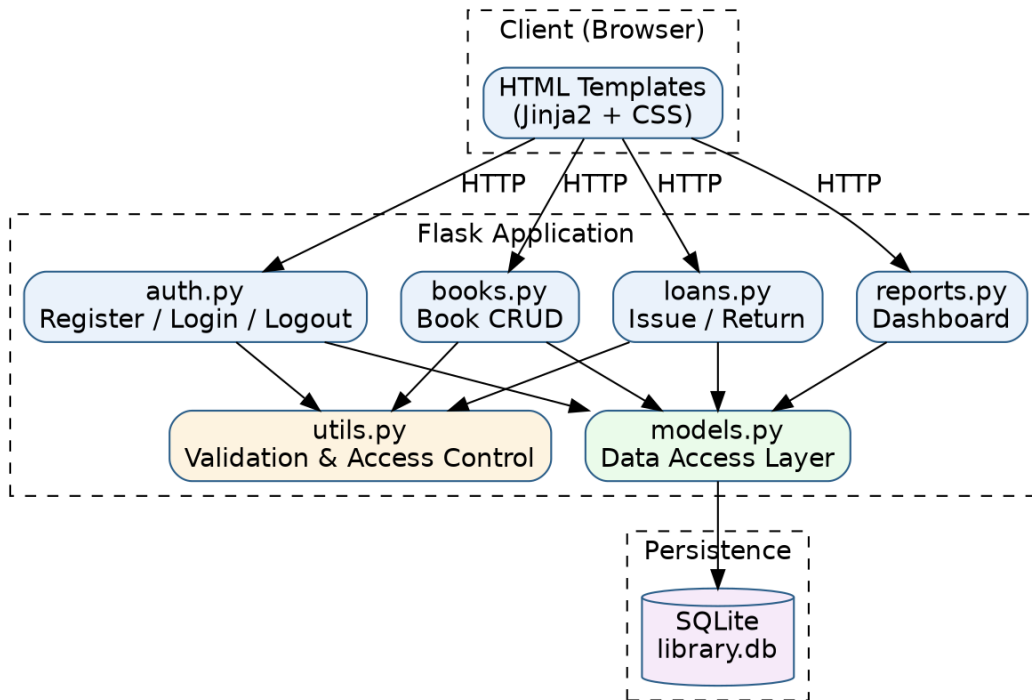


Figure 6.1 — System architecture diagram

7. Design Diagrams

7.1 Use Case Diagram

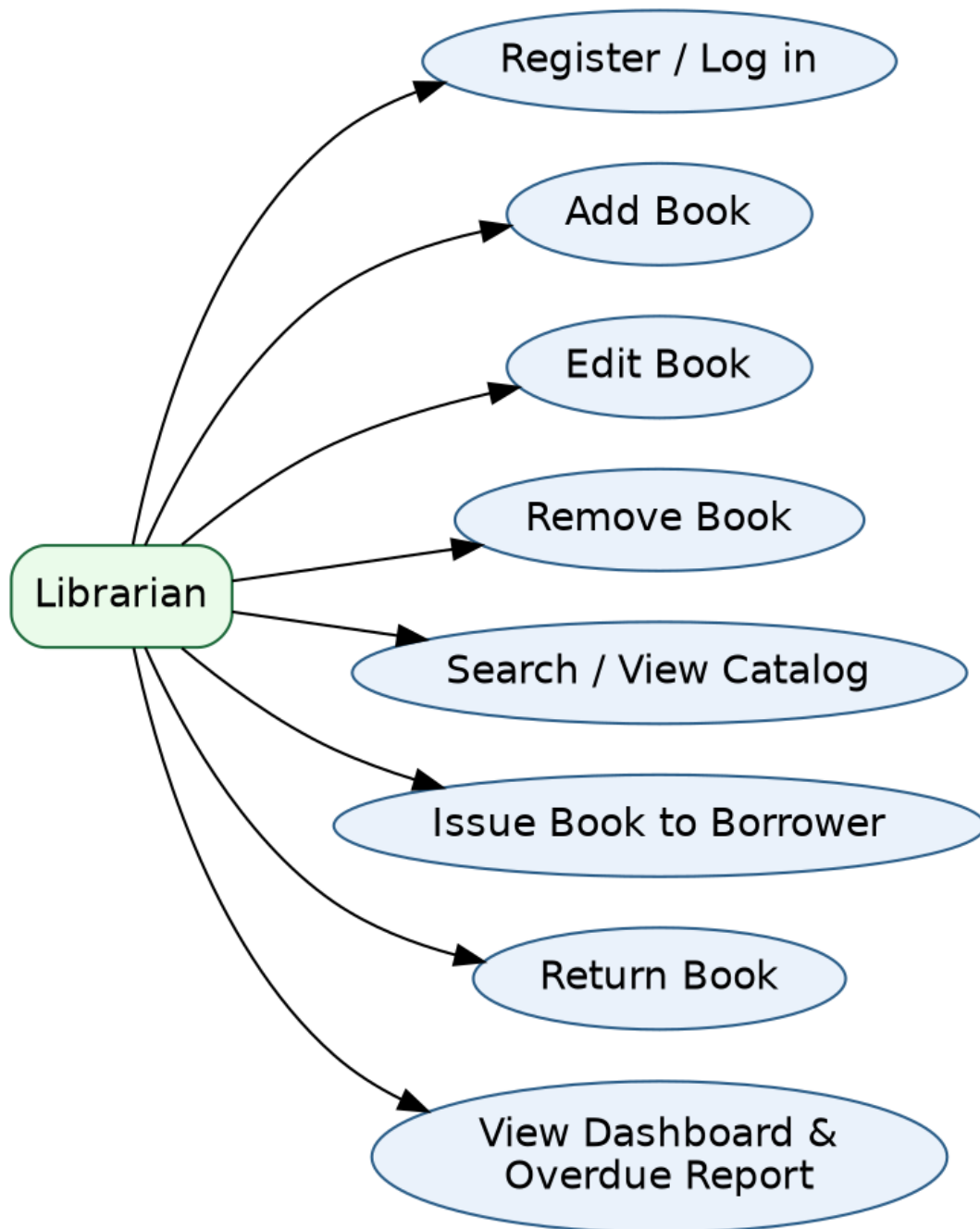


Figure 7.1 — Use case diagram

7.2 Workflow / Process Flow Diagram

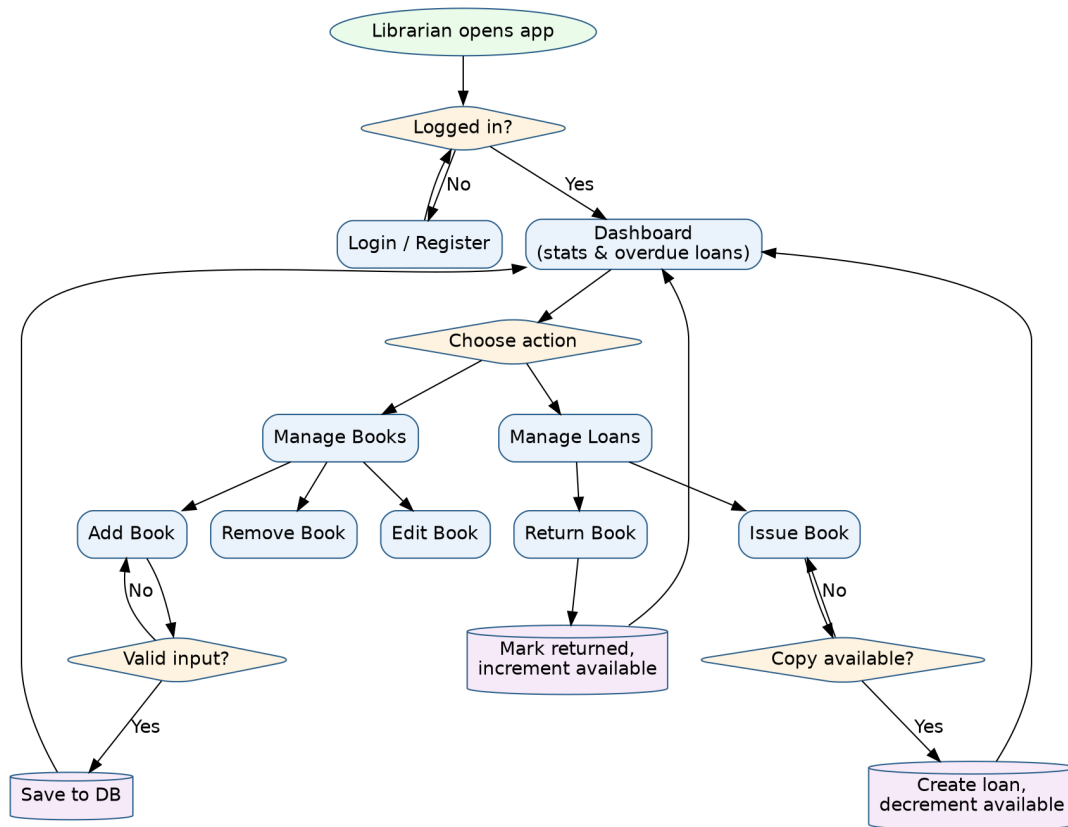


Figure 7.2 — Workflow diagram

7.3 Sequence Diagram — Issue Book

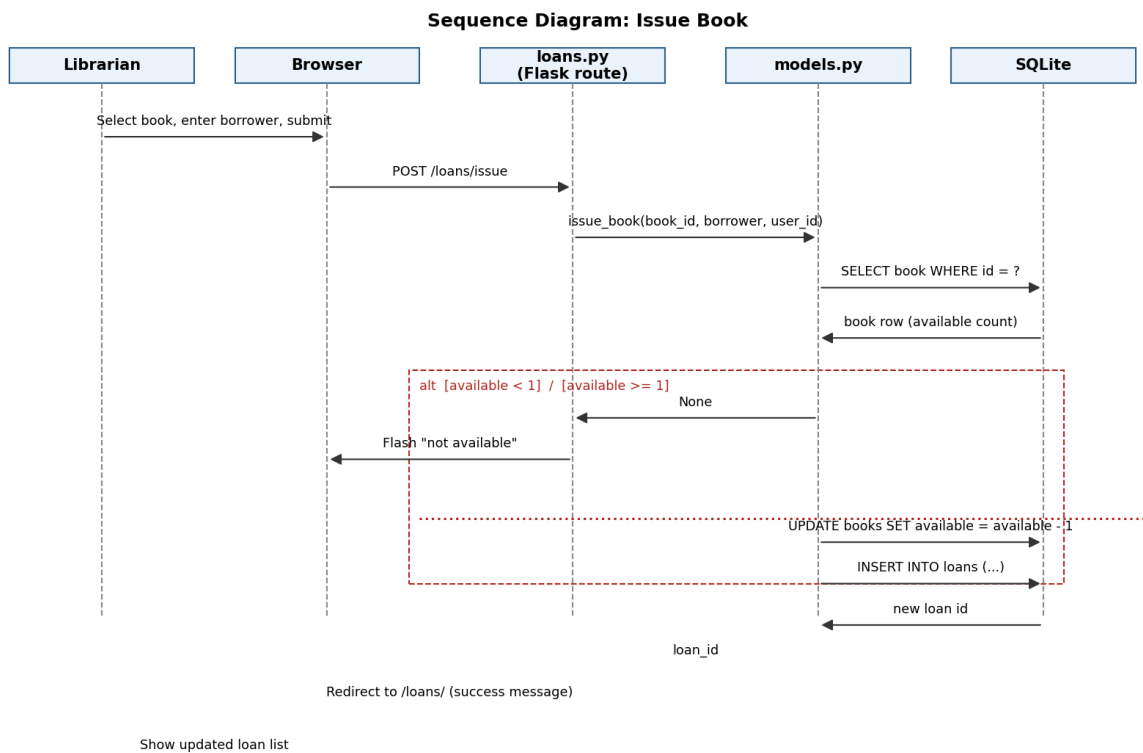


Figure 7.3 — Sequence diagram for the issue-book flow

7.4 Class / Component Diagram

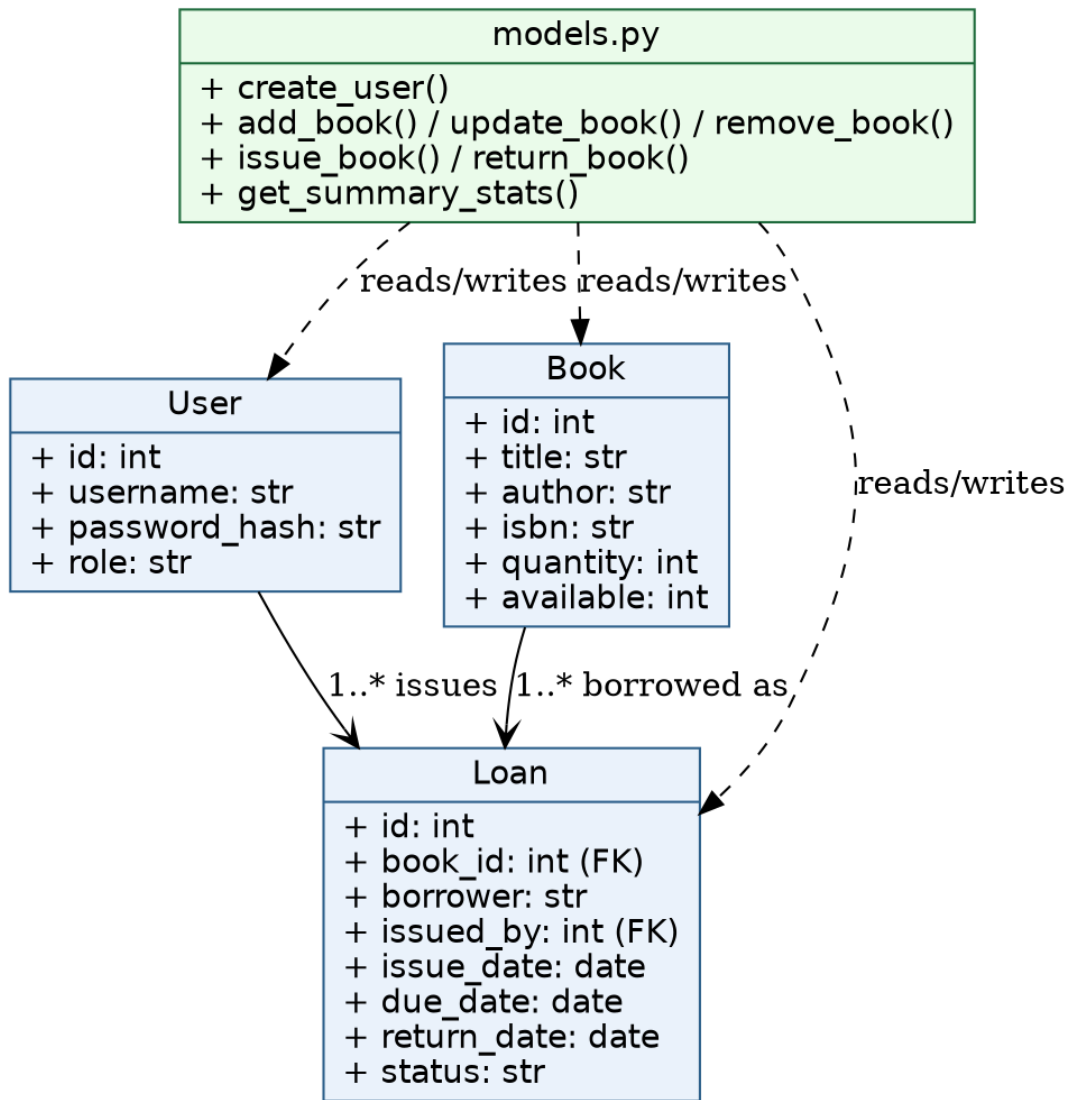


Figure 7.4 — Class / component diagram

7.5 ER Diagram

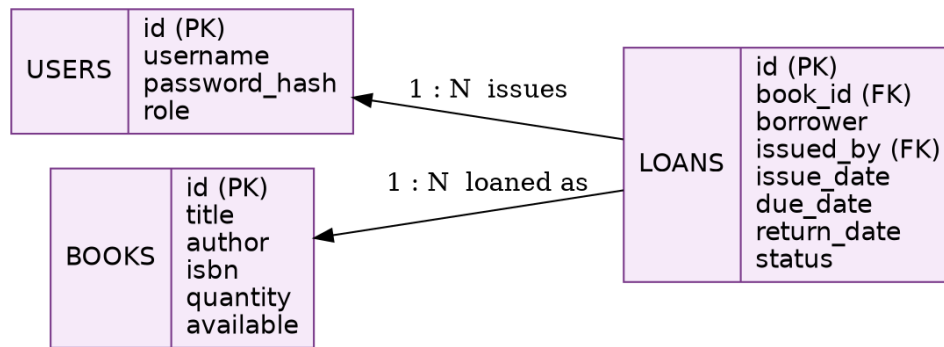


Figure 7.5 — Entity-relationship diagram

The executable version of this schema (with CHECK constraints and indexes) is in `schema.sql` at the project root.

8. Design Decisions & Rationale

- **Flask over a heavier framework** — the project's scope (a handful of CRUD flows plus auth) fits a micro-framework well and keeps the codebase easy to read for review.
- **Raw sqlite3 instead of an ORM** — writing SQL directly in a single data-access module (`models.py`) keeps the schema and queries explicit and transparent, which matters for a course project meant to demonstrate understanding of the underlying data operations.
- **Session-based auth with hashed passwords** — sufficient for a single-role (librarian) internal tool, while still avoiding the serious mistake of storing plaintext passwords.
- **Blueprint-per-feature structure** — `auth/books/loans/reports` are registered as separate Flask blueprints so each functional module can be read, tested, and extended independently.
- **Available vs. quantity as separate columns** — storing both lets the system answer "how many copies exist" and "how many are free right now" without recomputing it from the full loan history on every request.

9. Implementation Details

Project structure (9 Python modules, following a proper package layout):

```
app.py · config.py · db.py · schema.sql · models.py · auth.py · books.py · loans.py · reports.py ·
utils.py · templates/ · static/style.css · tests/test_app.py
```

- **app.py** — application factory; registers blueprints and error handlers.
- **db.py** — opens a request-scoped SQLite connection and initializes the schema on startup.
- **models.py** — every SQL statement in the project lives here (users, books, loans, reporting queries).
- **auth.py / books.py / loans.py / reports.py** — one Flask Blueprint per feature area.

- **utils.py** — a `ValidationError` exception, small validators (`require_non_empty`, `require_positive_int`), and the `login_required` decorator used across all protected routes.

Key implementation choices: the 14-day loan period is a named constant (`LOAN_PERIOD_DAYS`) in `models.py`; issuing a book is refused server-side (not just hidden in the UI) when available copies reach zero; and editing a book's total quantity proportionally adjusts its available count rather than overwriting it.

10. Screenshots / Results

The following are live screenshots of the running application (seeded with sample data).

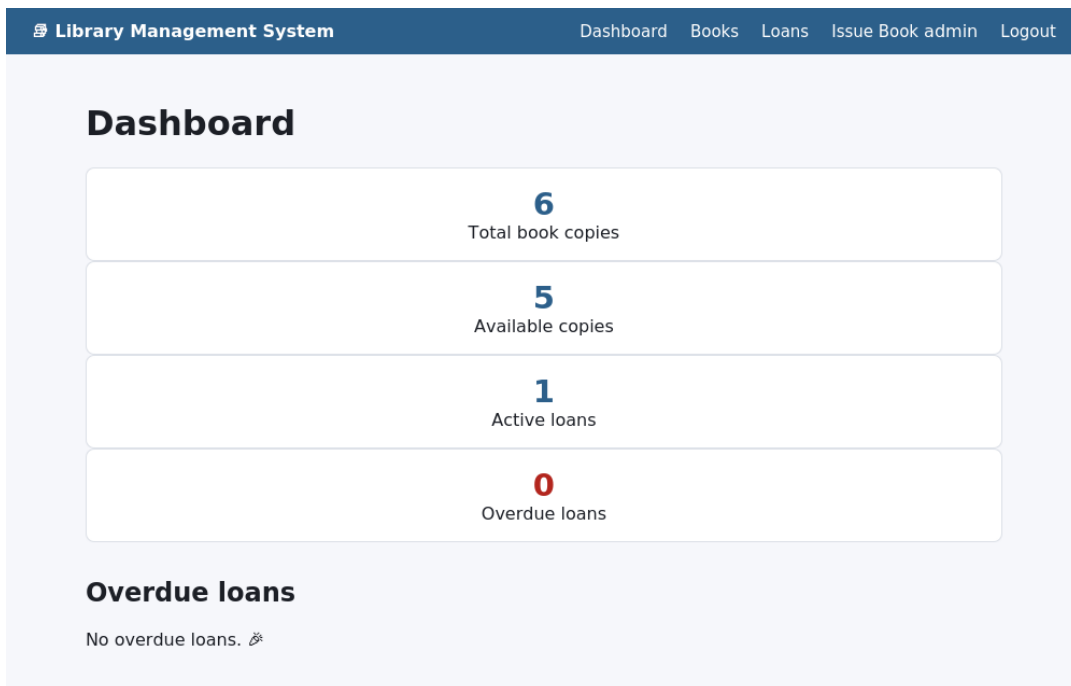


Figure 10.1 — Dashboard

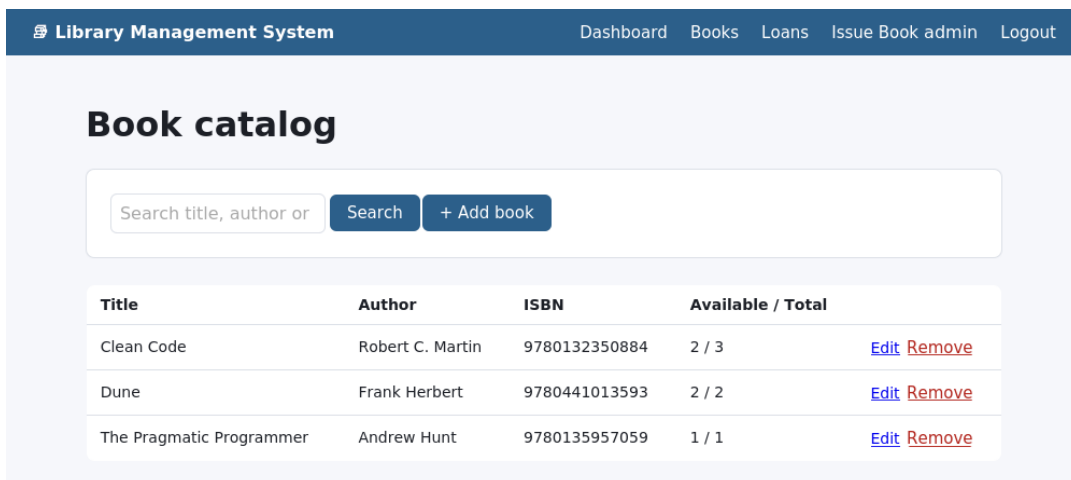


Figure 10.2 — Book catalog with search and CRUD actions

 Library Management System

[Dashboard](#) [Books](#) [Loans](#) [Issue Book admin](#) [Logout](#)

Issue a book

Book

Choose a book

Borrower name

Issue book

Figure 10.3 — Issue-book form

 Library Management System

[Dashboard](#) [Books](#) [Loans](#) [Issue Book admin](#) [Logout](#)

Active loans

Book	Borrower	Issued	Due	
Clean Code	Priya Sharma	2026-09-26 10:14:49	2026-10-10	Mark returned

Figure 10.4 — Active loans view

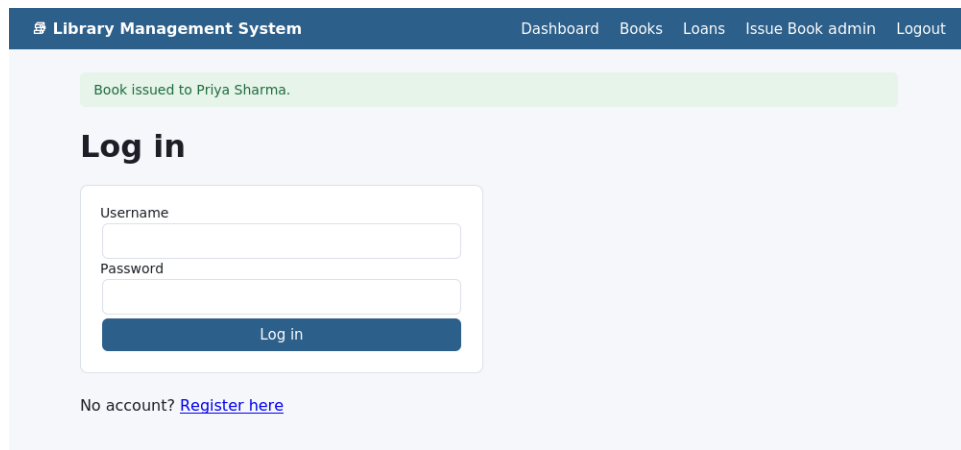


Figure 10.5 — Login page

11. Testing Approach

An automated test suite (`tests/test_app.py`) uses Python's built-in `unittest` module against a fresh, isolated temporary SQLite file per test case, so tests never touch the real database or affect each other. It covers authentication (successful login, wrong password, short password rejected, duplicate username rejected), access control (protected routes redirect anonymous users to login), book CRUD (add, list, reject empty title, remove), and the full lending cycle (issue decrements availability, return restores it, issuing an unavailable book is refused).

Test run output:

```
test_add_and_list_book (test_app.LibraryTestCase.test_add_and_list_book) ... ok
test_add_book_requires_login (test_app.LibraryTestCase.test_add_book_requires_login) ... ok
test_add_book_requires_title (test_app.LibraryTestCase.test_add_book_requires_title) ... ok
test_cannot_issue_unavailable_book (test_app.LibraryTestCase.test_cannot_issue_unavailable_book) ...
ok
test_dashboard_requires_login (test_app.LibraryTestCase.test_dashboard_requires_login) ... ok
test_duplicate_username_rejected (test_app.LibraryTestCase.test_duplicate_username_rejected) ... ok
test_issue_and_return_book (test_app.LibraryTestCase.test_issue_and_return_book) ... ok
test_login_with_wrong_password_fails (test_app.LibraryTestCase.test_login_with_wrong_password_fails)
... ok
test_register_and_login_success (test_app.LibraryTestCase.test_register_and_login_success) ... ok
test_remove_book (test_app.LibraryTestCase.test_remove_book) ... ok
test_short_password_rejected (test_app.LibraryTestCase.test_short_password_rejected) ... ok

-----
Ran 11 tests in 1.503s

OK
```

Result: **11 / 11 tests passing.**

12. Challenges Faced

- **Keeping available-copy counts consistent** when a book's total quantity is edited after some copies are already on loan — solved by adjusting available by the same delta as quantity, clamped at zero, instead of resetting it outright.

- **Avoiding raw-SQL duplication across routes** — solved by funneling every query through `models.py` so route handlers only call named functions.
- **Testing session-based, database-backed flows** without a heavyweight test framework — solved with a per-test temporary SQLite file plus Flask's built-in test client, avoiding any need for mocking.

13. Learnings & Key Takeaways

- Designing the schema (users, books, loans) before writing routes made the CRUD and lending logic straightforward to implement afterward.
- Centralizing validation and data access (`utils.py`, `models.py`) rather than inlining logic in each route made the codebase noticeably easier to test and to reason about.
- Writing tests against real HTTP requests (via Flask's test client) instead of only unit-testing helper functions caught issues in the actual request/response flow, such as redirect behavior for anonymous users.

14. Future Enhancements

- Member/borrower self-service accounts, separate from the librarian role.
- Automated email or SMS reminders as a loan's due date approaches.
- Pagination and CSV export for large catalogs and reports.
- Fines/penalties tracking for overdue returns.

15. References

- Flask documentation — <https://flask.palletsprojects.com/>
- Python sqlite3 module documentation — <https://docs.python.org/3/library/sqlite3.html>
- Werkzeug security utilities — <https://werkzeug.palletsprojects.com/en/latest/utils/#module-werkzeug.security>
- Mermaid diagram syntax reference — <https://mermaid.js.org/>